



ZOMBIE ZERO

TECHNICAL DEMO



PROJECT GOALS

-
- Create a production ready game deployed online
 - Should be fun to play
 - No proof on concept, an actual game
 - Playable by everyone (Cross Platform)

TEAM DISTRIBUTION

-
- Muhammad Aniq Nasir
 - [GitHub Profile](#)
 - Worked on Core Game Logic, Core Game Systems ...
 - Hasham Tanveer
 - [GitHub Profile](#)
 - Worked on Game UI, High Scores and Weapon Systems ...
 - Muhammad Mujtaba
 - [GitHub Profile](#)
 - Worked on Game Engine, Enemy Spawning ...

GAME OVERVIEW



Genre

Top-Down Shooter



Objective

Clear all zombies in the city



Availability

Windows, MacOS, Linux*

*Linux executable is not available as of now, but gradle setup has been completed

ARCHITECTURE OVERVIEW



Use Java as main Language



Uses JavaFX's FXML & CSS for UI



Built using a Custom Game Engine using JavaFX

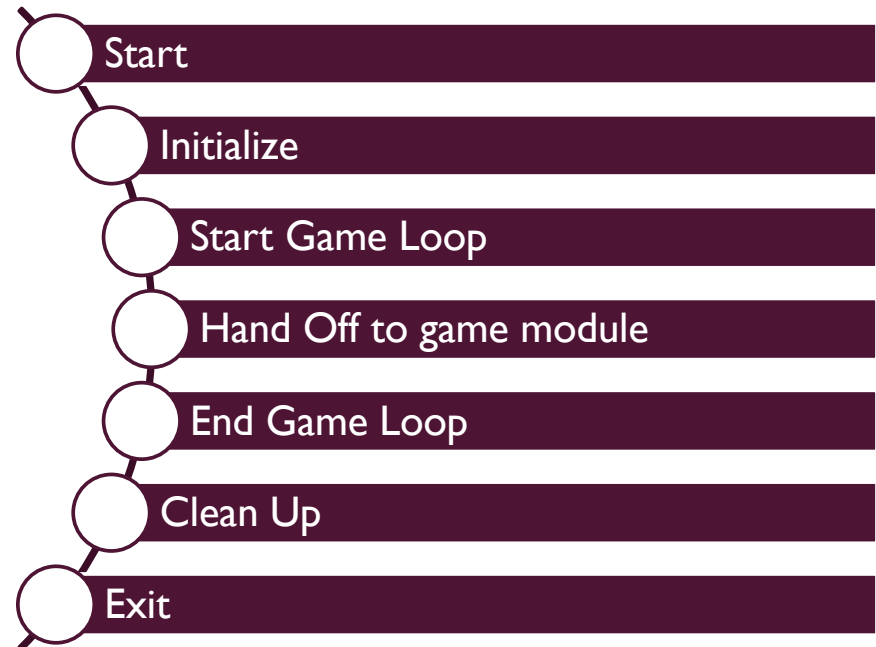
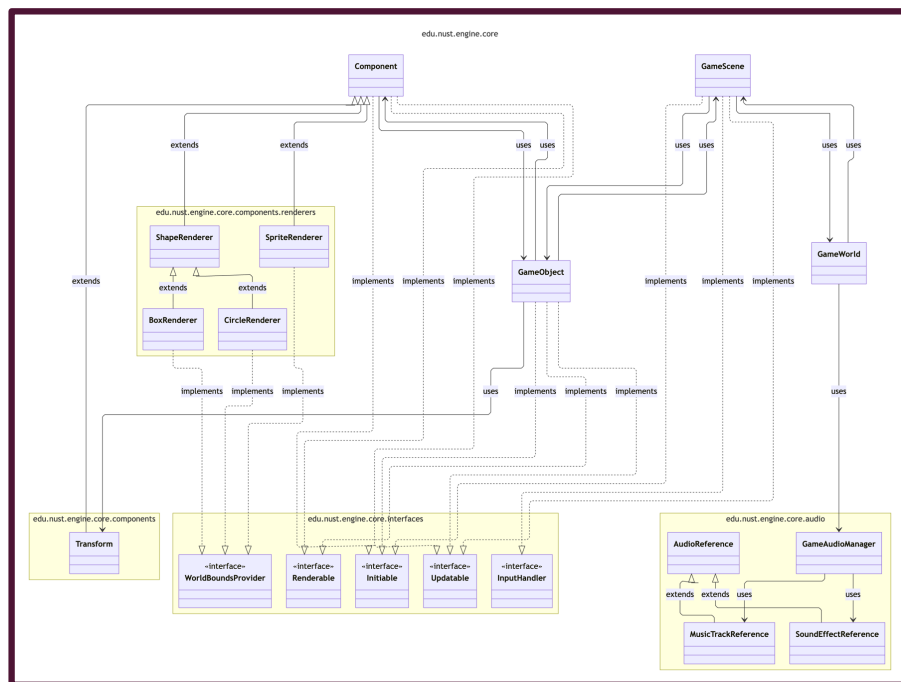


Architecture heavily inspired from Unity Game Engine



Consists of two separate independent modules; Engine & Game

ENGINE HIGH LEVEL OVERVIEW



INDEX

Engine		Game Mechanics		Scenes Breakdown	
GameWorld	9	Asset Manager	33	<u>StartScene</u>	57
<u>GameScene</u>	10	Collision Manager	34	<u>LevelScene</u>	58
GameObject	15	Player	37	<u>HighScoresScene</u>	63
Component	16	Enemy	40		
Rendering Pipeline	22	<u>Path finder</u>	45		
Input Handling	23	Weapon System	48		
DevConsole	24	Health Bar	52		
Resources Management	27	Audio Management	53		
Math Library	28	Gradle Tasks	54		
GameLogger	29	UML Generation Pipeline	55		
Audio Management	30				



ENGINE

DEEP DIVE



GAMEWORLD

- GameWorld owns the JavaFX Stage and Scene
- Drives the AnimationTimer loop for main Game Loop
- Delegates each frame to the active GameScene
- Switches scenes by swapping root children (world + UI)

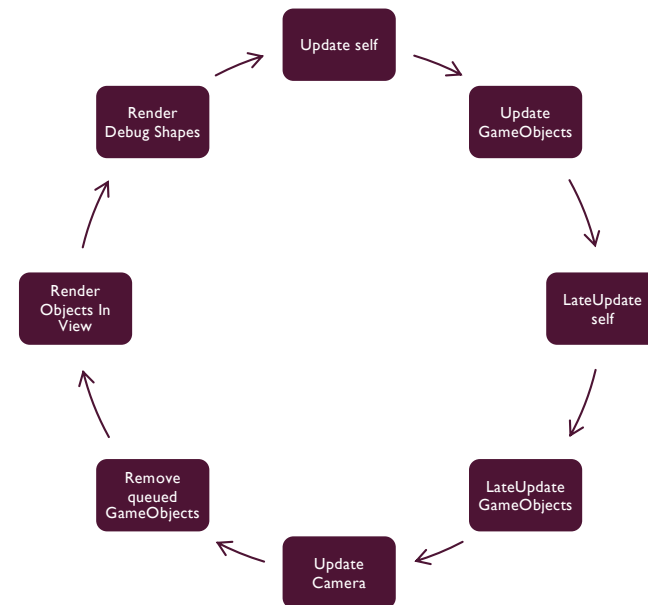
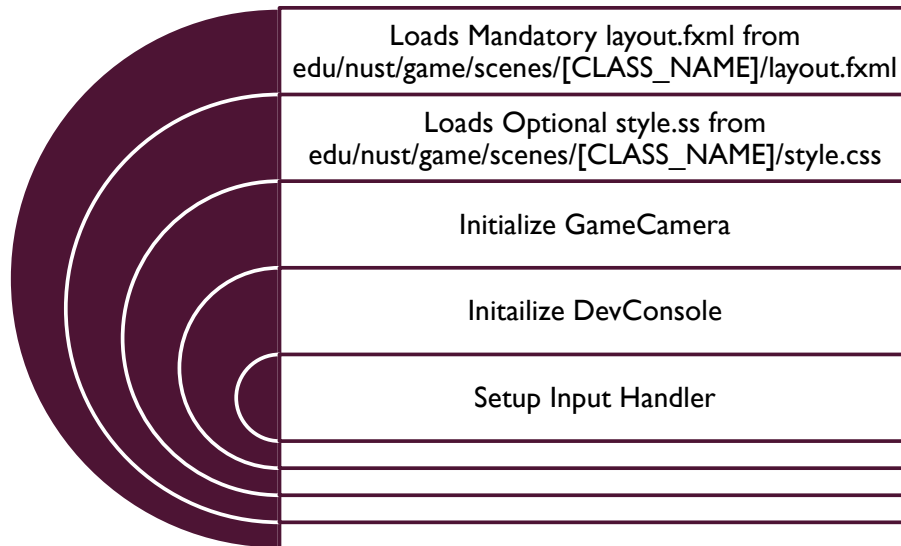
GAMESCENE

- Represents a single scene in the game
- Implements Initiable, Updatable
- Owns the game object list, the world canvas, and the FXML UI
- Has separate layers for GameObject rendering, UI Rendering, DevConsoleRendering
- Abstracts input hooks from JavaFX into overridable methods.

GAMESCENE (LIFECYCLE)

- Init
 - Loads Mandatory layout.fxml from edu/nust/game/scenes/[CLASS_NAME]/layout.fxml
 - Loads Optional style.ss from edu/nust/game/scenes/[CLASS_NAME]/style.css
 - Initialize GameCamera
 - Initailize DevConsole
 - Setup Input Handler
- Game Loop (Received from GameWorld)
 - Update self
 - Update GameObjects
 - LateUpdate self
 - LateUpdate GameObjects
 - Update Camera
 - Remove queued GameObjects
 - Render Objects In View
 - Render Debug Shapes

GAMESCENE (LIFECYCLE)



GAMESCENE (GAMECAMERA)

- Can change camera zoom
- Can follow a given position
- For Movement the Canvas is moved, thus all GameObjects moves when camera is moved
- Can Shake
 - Depends upon given intensity
 - Consecutive calls increase shake intensity
- Working
 - Stores 2 positions (actual position and render position)

GAMESCENE (RENDER ORDER AND CULLING)

- Objects are sorted by renderLayer and culled by camera bounds.
- Object bounds are provided by renderer components using WorldBoundsProvider interface
- renderLayer by default is 0, if two objects has same renderLayer, then the one added last takes precedence for rendering

GAMESCENE (DEBUG RENDERING)

- Grid, mouse location, and timed/frame debug shapes can be rendered on the world canvas on top of all gameobjects.
- Grid renders a grid, was used in blank scenes when scene background hadn't created
- Mouse Location shows a red crosshair at the mouse, highlighting the world coordinates of that point
- Timed shapes stored in a set and cleared after the given TimeSpan
- Frame-only shapes cleared each frame

GAMEOBJECT

- Represents any and all objects in the game
- Implements Initiable, Updatable and Renderable
- Can be added or removed at runtime
- Composes of Components with their own lifecycle
- Can have multiple Tags
 - A Tag is simply a class extending the Tag class, the class itself is used as a tag rather than a string

COMPONENT

- Represents behavior of a GameObject
- Implements Initiable, Updatable and Renderable
- Can be added or removed at runtime
- Can be added to multiple different GameObjects, but only one instance can exist on a GameObject at a given time

COMPONENT (TRANSFORM - PRE EXISTING)

- Guaranteed to be present on any GameObject
- Used to represent position, rotation and anchor of this object
- Anchor can be set using a Vector2D, predefined helpers for setting anchor to center, top left, bottom, center etc.

COMPONENT (TRANSFORM - PRE EXISTING)

- Guaranteed to be present on any GameObject
- Used to represent position, rotation and anchor of this object
- Anchor can be set using a Vector2D, predefined helpers for setting anchor to center, top left, bottom, center etc.

COMPONENT (BOXRENDERER - PRE EXISTING)

- Renders a Box with the given size
- Position is fetched from Transform component of attached GameObject
- Can be rotated as well
- Is affected by anchor positioning
- Can be filled, stroked
- Corner radius can be applied
- Properties can be changed at runtime

COMPONENT (CIRCLERENDERER - PRE EXISTING)

- Renders a circle with the given radius
- Position is fetched from Transform component of attached GameObject
- Is affected by anchor positioning
- Can be filled, stroked
- Properties can be changed at runtime

COMPONENT (SPRITERENDERER - PRE EXISTING)

- Draws a sprite or sprite sheet with the given frame counts
- Position is fetched from Transform component of attached GameObject
- Can be rotated and affected by anchor positioning
- Supports tint and opacity
- Supports Animation by setting animation time, after the given time the frame is automatically changed
- Can be flipped
- Properties can be changed at runtime

RENDERING PIPELINE

- Handled by GameScene
- worldLayer is a Canvas
 - Renders GameObjects
- uiLayer loaded from FXML and CSS
 - Renders JavaFX nodes
- consoleLayer stacked on top
 - Renders DevConsole, above everything

INPUT HANDLING

- Handled in GameScene by overriding methods given in InputHandler interface
- Has the following events
 - onKeyPressed
 - onKeyReleased
 - onMousePressed
 - onMouseReleased
 - onMouseMoved
 - onMouseDragged

DEVCONSOLE

- Can be opened in any scene by pressing “Shift + `”, and exit by pressing “Esc”
- Commands to modify game behaviour at runtime
- Supports suggestions and history
- Displays the following stats
 - Game FPS
 - Total Objects added
 - Objects being Rendered (objects not visible are culled)

DEVCONSOLE (COMMAND)

- Commands stored in a map with normalized names
- Executors return strings shown in console output
- Every GameScene can register commands that can be executed in that scene
- Shows suggestion with command usage realtime, can press tab to autocomplete the command
- Preserves History of Previously Executed Commands in this session
- Show output of previos commands as well

DEVCONSOLE (DEFAULT COMMANDS)

- `/help`
 - Shows Registered Commands
- `/debugGrid [true|false|number|empty]`
 - Toggle grid, with given size if number specified
- `/debugMouseLocation [true|false|empty]`
 - Toggle world mouse crosshair + coordinates.
- `/setCameraZoom [number]`
 - Set camera zoom
- `/toggleUI [true|false|empty]`
 - Toggle FXML UI visibility (excluding dev console).

RESOURCES MANAGEMENT (RESOURCES)

- Path for resources are resolved relative to `/edu/nust/game/` in resources directory
- resources directory is bundled with the executable by Java
- Supports optional Resources and mandatory resources by `tryGetResource(...)` and `getResourceOrThrow(...)` respectively
- Utility for loading a JavaFX Image from the given path, assumes loading image is mandatory
- Used extensively to get every kind of resource including, text, image, or any other type

MATH LIBRARY

- Primitive classes to represent positions, velocity, time, areas etc.

- All of Below has methods for arithmetic operations and more

- Contains

- Vector2D

- A container for 2 numbers
 - Used to represent positions, velocities etc.
 - D stands for double, represents the data type used for storing numbers

- TimeSpan

- Used to represent a said amount of time
 - Used instead of int, double etc. to remove confusion regarding unit of time
 - Constructor is privatised, instead factory methods are used that explicitly mention the unit, e.g `TimeSpan.FromSeconds(10)`.
 - Getting the time value also require explicit unit e.g. `deltaTime.asMilliseconds()`
 - Has methods for interconversion between units ranging from nanosecond to day
 - Internally stores the time in the form of nanoseconds

- Rectangle

- Used to represent an area covered by a Rectangle
 - Is Axis Aligned, i.e. cannot be rotated
 - Can be shrunk and grown
 - Queries for intersection between rectangles, containing a given point etc.

- Angle

- Used to represent a certain angle
 - Constructor is privatised, instead factory methods are used that explicitly mention the unit, e.g `Angle.fromDegrees(10)`.
 - Getting the time value also require explicit unit e.g. `rotation.getRadians()`
 - Has methods for interconversion between radians and degrees

GAMELOGGER

- Wraps SLF4J Logger with custom color prefixes and formatting for each level
- Writes each log to its own file in logs/latest.log, and archives the old logs based on timestamp logs/archived/
- Logging disabled in packaged versions
- Logs can be filtered by one global level
- Level Types in order:
 - Trace
 - Intended to represent function calls to visually trace the program
 - Debug
 - Intended for debug only
 - Info
 - Default Log Type
 - Success
 - Intended to represent a successful operation
 - Warn
 - Intended for warnings, i.e. optional things not loaded
 - Error
 - Can throw or not based on user preference
 - LogProgress
 - Intended to represent a progress that can be started, reported on, and ended
 - Gives color to the prefix

AUDIO MANAGEMENT

- AudioReference stores the audio URL and common metadata
- Created internally by GameAudioManager of GameScene
- Two Types
 - SoundEffectReference
 - Represents short bursts of sounds like bullet shots etc.
 - Cannot be paused or have granular control, but can have overlapping instances of same reference
 - Uses JavaFX Clip internally
 - MusicTrackReference
 - Represents long sounds, musics mainly
 - Can be paused and has granular control, but cannot have overlapping instances of same reference
 - Uses JavaFX MediaPlayer internally

AUDIO MANAGEMENT (GAMEAUDIOMANAGER)

- Manages all in-game audio
- Can load and unload an AudioReference
- Controls Global Volume for muting, same global volume for both Effects and Tracks
- Linked to the GameWorld
- Loaded Effects are only unloaded when game exits
- AudioReferences persists when switching scenes
- All loaded references are cached by full relative path; loading the same asset twice returns the existing reference
- All paths are resolved relative to edu/nust/game/assets/audio/



GAME MECHANICS

DEEP DIVE



ASSET MANAGER

- Singleton that loads assets once and reuses them
- Uses an enum interface for Asset Type
- Has a cache of loaded images
- Used to loads images for character, enemy and boss etc.

COLLISION MANAGER

- Collects Damageable, Damaging, and Concrete objects and resolves collisions
- Two types of Collision
 - Concrete (Concrete Objects & Concrete Objects)
 - Damaging (Damaging Objects & Damageable Objects)
- Notifies the objects that the collision has occurred with reference to other

COLLISION MANAGER (INTERACTIONS)

- Stops bullets (Bullets collision with walls), Player movement restrictions (Concrete & Concrete)
- When Bullet hits enemy, increase the follow radius of the enemy (Damaging & Damageable)
- Award kill score when an enemy dies
 - Separate for each enemy type



COLLISION MANAGER (HITBOX)

- Performs AABB collision checks
- Supports debug visualization
- Separate methods for collision with each sides of the AABB rectangle



PLAYER

- Extends Character and Implements Damageable and Concrete
- Movement speed: 50 units/sec
- Takes input from WASD and Arrow keys
- Injects a walkabilityChecker backed by the level's collision mask for movement validation
- Camera follows the player's position every frame in lateUpdate()

PLAYER (ANIMATION)

- Sprite sheets
 - Idle
 - Run in 4 directions
- Animated at 75ms per frame
- Direction is determined from the movement delta each frame
- Idle animation plays when no input is held



PLAYER (HEALTH)

- Health is a separate class shared by both player and enemies
- Tracks current and max health
- Red tint is blended onto the sprite as HP drops
 - $\text{tintStrength} = (1.0 - \text{healthRatio}) \times 0.5$
- HitBox is smaller than visual sprite

ENEMY (ABSTRACT)

- Implements Concrete and Damageable
- Subclasses must have methods for attacking and animations
- Per-frame state machine
 - if dead → play death animation
 - if player out of range → idle/melee
 - if in range → pathfind and move
- Path re-computed every 300ms via Pathfinder
- Default follow range is 200 units
- When a bullet hits, follow range doubles

ENEMY (BASE ENEMY)

- Extends Enemy class
- Standard Enemy spawned across the map
- Spawns a BasicAttackObj on melee
 - Hitbox at current location lasting 400ms
- Death
 - plays 6-frame death sheet
 - then opacity decrease at the rate of -0.05 per frame until 0
 - then destroys itself

ENEMY (MINIBOSS)

- Extends Enemy class
- Spawns at bottom middle of the first section of level
- Has 2 attacks
 - Standard melee
 - Ranged axe throw
- Axe throw
 - Range 300 units
 - Spawn

ENEMY (BOSS)

- Extends Enemy class
- Spawns at top right of the second section at the end of the level
- Same base stats as MiniBoss
- Upgraded version of the MiniBoss
- Has a twist (~Spoilers)



ENEMY (HEALTH)

- Health is a separate class shared by both player and enemies
- Tracks current and max health
- Red tint is blended onto the sprite as HP drops
 - $\text{tintStrength} = (1.0 - \text{healthRatio}) \times 0.5$
- HitBox is smaller than visual sprite

PATH FINDER (INITIALIZATION)

- Uses MapNodeSetter class
- First creates a grid of nodes using native fixed size 2D array in MapNodeSetter
- Loops over the the Level Collision Mask to determine walkable area
- A* search ready to calculate path from any given point to another one

PATH FINDER (A*)

- Nodes:
 - Nodes are points on the grid.
 - Each node has gcost,hcost and fcost.
 - Each node can be open (open for evaluation), checked, solid or can be start or goal node
 - gCost is the distance of node from the start node
 - hCost is the distance of node from Goal Node
 - fCost is the sum of hCost and fCost

PATH FINDER (A*)

- First sets up the start (Enemy Position) and the goal node (Player Position)
- From the start node (setting it as current node) it evaluates its adjacent nodes and adds them to the open List (Queue with fcost as its comparator, i.e constructs a list with smaller fCost at the top)
- Setups the gCost ,hCost and fCost of the nodes
- Set current node to be the first element of priorityQueue (the node with least amount of fcost)
- Check the current node and evaluates its neighbours.
- Neighbour is only be added to open list if it is not start node, not solid node, not already a open node or it has greater gCost than the current node's gcost+1
- If a neighbouring node is added to the openList its parent is set as current node for which it was evaluated.
- Now it continues this till it reaches the goal or does not find the goal within 3000 iterations.
- To return the path we trace back using the parent nodes. If no path was found empty list/path is returned.

WEAPON SYSTEM (WEAPON)

- Added to the scene alongside the player as a `GameObject`
- Orbits the player at a fixed offset, aimed toward the mouse cursor
- Sprite flips vertically when aiming left ($\text{angle} > 90^\circ$)
- Uses a 6-frame idle sprite sheet with animation
- Plays a 3-frame muzzle flash rendered at the barrel tip on fire
- Supports auto-fire (hold mouse button)
- `fireWeapon()` is called every frame; enforces fire rate via a cooldown ($1 / \text{fireRate}$ seconds per shot)

WEAPON SYSTEM (AMMO)

- Ammo is a standalone class (not a Component) owned by Weapon
- Max ammo is 30
- Decrement by 1 per shot; automatically triggers reload when it reaches 0
- Manual reload can also be triggered via the R key
- Reload has two phases: a 0.5s delay, then a 2.0s fill
- During reload, firing is blocked

WEAPON SYSTEM (BULLET)

- Bullet is a `GameObject` implementing `Damaging` and `Concrete`
- Spawned at the player center position, directed toward the mouse
- Travels at a speed of 250 units/second; has a max range of 1000 units
- Destroys itself when out of range
- Carries a damage value set by `Weapon`
- Implements `triggerCollisionEffect()`
 - Play a random impact sound
 - Spawns a `BulletImpact` particle (50% of the time)
 - Then destroys itself
- Will not damage `Player`

WEAPON SYSTEM (UI)

- AmmoBar is a JavaFX HBox placed in the FXML UI layer
- Displays 10 bullet icon slots; each slot represents 3 rounds (30 max / 10 slots)
- Each slot shows either a filled or empty bullet sprite (bullet.png / bullet_empty.png)
- IMAGE:Ammo Bar Image



HEALTH BAR

- JavaFX UI Node that is rendered in the FXML UI layer
- Extends Bar; provides a slider than can be filled by a given amount
- Renders HP-Bar.png as a frame and crops HP.png fill by the current health ratio
- Crop width shrinks as health decreases

AUDIO MANAGEMENT IN GAME

- Audios
 - Exposes methods for sound effects and music references
 - Randomized sound effect selection
 - Usage: `Audios.clickNeutralRef().ifPresent(SoundEffectReference::play)`
- MusicManager
 - Uses Music references from Audios
 - Transitions to the music of the given scene
 - Does by fading out the old one, and fading in the new one

GRADLE TASKS

- packageWindows
 - Packages into an exe installer
- packageMac
 - Packages into an dmg installer
- packageLinux
 - Packages into linux installer
- buildExecutableJar
 - Builds a jar that can be opened by JRE
- run
 - Runs the application with full logging (Maximum log level defined in Main class)
- runNoLogs
 - Runs the application with no logging
- runTestLogger
 - Runs a test that tests the logger of all formats, error message etc

UML GENERATION PIPELINE (MISC.)

- Requires Python & Mermaid CLI in npm
- Generate UML using a python script
- Uses mermaid for to scaffold structure, then renders using the mermaid renderer
- AI Generation Disclosure



SCENES

BREAKDOWN OF EACH SCENE





STARTSCENE

- Main entry point of the game
- Routes to main game and high scores along with exit game button
- IMAGE: show start screen



LEVELSCENE

- The main level of the game
- Objective is to clear out all zombies
- IMAGE: Show normal gameplay image

LEVELSCENE (COLLISION MASK)

- Defines a list of rectangles that are used to defined unwalkable areas
- IMAGE 1: showing code ss of a rectangle start point
- IMAGE 2: showing that point in game using dev console

LEVELSCENE (ENEMY SPAWNING)

- Spawns when the game is loaded
- Generates spawn points across a grid with random offsets and validation.
- Avoids non-spawnable area
- Validates with collision mask and movement checks

LEVELSCENE (OBJECTS PLACEMENT)

- Static objects are loaded from objects_placements.txt and added to the scene
- objects_placements.txt generated once in development
- IMAGE 1: Show file
- IMAGE 2: Show object placed

LEVELSCENE (WIN AND LOSE SCREEN)

- Score Text:
 - Displays score as 3-digit number (7 displays as 007)
 - Uses a spritesheet for the number font
- IMAGE 1: win screen
- IMAGE 2: lose screen

HIGHSCORESCENE

- Shows the high scores fetched from `highscores.txt`
- Loads and renders top 5 scores within a list.
- IMAGE: Show high score screen

HIGHSCORESCENE (STORAGE)

- Stored in csv like format in highscore.txt
- Name of current player stored in PlayerSession
- Scoring breakdown:
 - 1 point every 5 seconds
 - 3 for regular zombies
 - 8 for miniboss
 - 15 for boss



OOP CONCEPTS USED

SOLID & OTHER PRINCIPLES



SOLID (SINGLE RESPONSIBILITY PRINCIPLE)

- Resources only resolves and loads assets.
- Score only tracks score state and time-based increments.
- Health holds current/max HP and ratio logic.
- HealthBar renders health bar by a given.
- CollisionManager detects collisions and calls Health.takeDamage(x); it does not know how UI renders health.

SOLID (OPEN CLOSED PRINCIPLE)

- New behaviors can be added by creating Component subclasses without changing Component itself.
- New scenes extend GameScene without altering engine code.
- BasicEnemy, MiniBoss, and Boss all extend Enemy without modifying any enemy base logic.

SOLID (LISKOV SUBSTITUTION PRINCIPLE)

- `GameScene.addObject(GameObject)` accepts any subclass (Player, Weapon, Bullet).
- `CollisionManager` holds lists of `Damageable` and `Damaging` — any `Player` or `Enemy` subclass can be passed without the manager needing to know the concrete type.

SOLID (INTERFACE SEGREGATION)

- Initiable, Updatable, Renderable, InputHandler are separated.
 - GameObject implement Initiable, Updatable, Renderable but not InputHandler.
 - Similarly, GameScenes implement Initiable, Updatable, InputHandler but not Renderable
- Bullet implements only Damaging and Concrete; it is never forced to implement Damageable since bullets cannot receive damage.

SOLID (DEPENDENCY INVERSION PRINCIPLE)

- GameScene's culling uses WorldBoundsProvider interface, decoupling from concrete renderers.
- CollisionManager depends entirely on the Damageable, Damaging, and Concrete interfaces — it never imports Player, Enemy, or Bullet directly.

COMPOSITION & AGGREGATION

-
- GameScene composes GameCamera
 - Destroying GameScene destroys GameCamera
 - GameWorld Aggregates GameScene
 - Scenes can exist after destroying GameWorld

CONCLUSION

FINAL REMARKS

TECHNICAL CHALLENGES

-
- How to convert the map to a grid and the path obtained from grid back to map coordinates?
 - How to place thousands of objects in a single scene?

FUTURE ROADMAP

-
- Add more levels
 - Add More Bosses
 - Add Lore
 - Add Campaign
 - Add Multiplayer
 - Improve Game Engine Versatility
 - Improve Enemy AI
 - Improve UI
 - Improve Game Design



Game



Source (Github)

THANK YOU!

TRY IT OUT!



Slides

Q & A

ANY QUESTIONS, IF SO, FEEL FREE TO ASK?